

MongoDB: Diseño del Modelo de Datos

Bases de Datos II



Instituto Tecnológico
de Buenos Aires

Introducción

- Los modelos de datos efectivos satisfacen las necesidades de su aplicación. La consideración clave para la estructura de sus documentos es la decisión de **embeberlos o utilizar referencias**.

Enfoque de documentos embebidos

- Con MongoDB, puede embeber datos relacionados en una única estructura o documento
- Estos esquemas generalmente se conocen como modelos "desnormalizados" y aprovechan los documentos de MongoDB
- Considere el siguiente diagrama:

Esquema de ejemplo

```
{  
  _id: <ObjectId>,  
  username: "123xyz",  
  contact: {  
    phone: "123-456-7890",  
    email: "xyz@example.com"  
  },  
  access: {  
    level: 5,  
    group: "dev"  
  }  
}
```

Embedded sub-
document

Embedded sub-
document

Enfoque de documentos embebidos

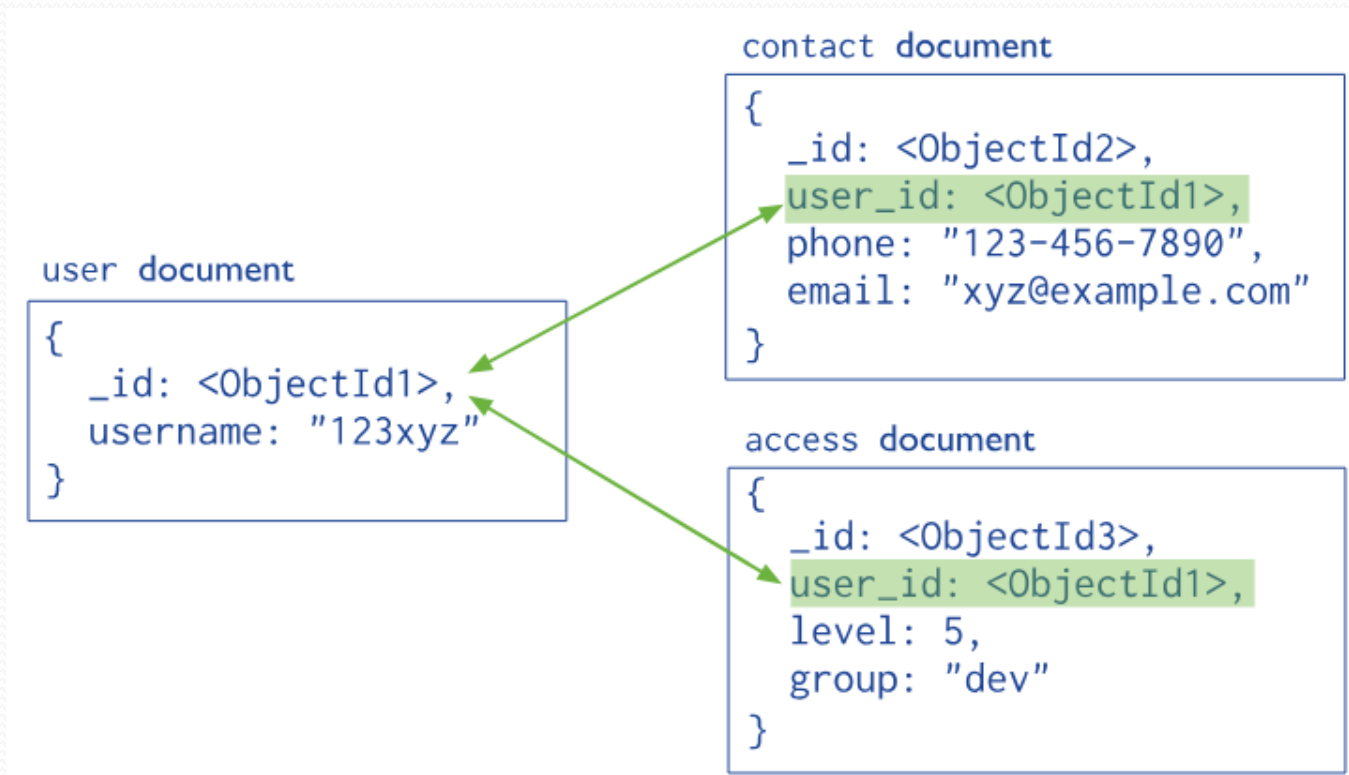
- Los modelos de datos embebidos permiten que las aplicaciones almacenen información relacionada en el mismo registro de la base de datos. Como resultado, es posible que las aplicaciones deban emitir menos consultas y actualizaciones para completar las operaciones comunes
- En general, utilice modelos de datos incrustados cuando:
 - Tenga relaciones "contiene" entre las entidades
 - Tenga relaciones de uno a muchos entre entidades. En estas relaciones, los "muchos" o documentos secundarios siempre aparecen con o se ven en el contexto de "uno" o documentos principales
- En general, los documentos embebidos proporcionan un mejor rendimiento para las operaciones de lectura, así como la capacidad de solicitar y recuperar datos relacionados en una sola operación de base de datos. Los modelos de datos embebidos permiten actualizar los datos relacionados en una sola operación de escritura atómica

Documentos BSON

- El tamaño máximo del documento BSON es de 16 megabytes
- MongoDB no admite más de 100 niveles de anidamiento para documentos BSON

Modelo de Datos Normalizado

- Los modelos de datos normalizados describen relaciones usando referencias entre documentos



Modelo de Datos Normalizado

- En general, utilice modelos de datos normalizados:
 - cuando la incorporación resultaría en una duplicación de datos pero no proporcionaría suficientes ventajas de rendimiento de lectura para superar las implicaciones de la duplicación
 - para representar relaciones más complejas de muchos a muchos
 - para modelar grandes conjuntos de datos jerárquicos
- Las referencias proporcionan más flexibilidad que el modelo embebido. Sin embargo, las aplicaciones del lado del cliente deben emitir consultas de seguimiento para resolver las referencias. En otras palabras, los modelos de datos normalizados pueden requerir más viajes de ida y vuelta al servidor

Modelo de relaciones *One-to-One* con documentos embebidos

- Considere el siguiente ejemplo que mapea las relaciones de usuario y dirección
- El ejemplo ilustra la ventaja de embeber sobre las referencias si necesita ver una entidad de datos en el contexto de la otra
- En esta relación de *one-to-one* entre el usuario y los datos de la dirección, la dirección pertenece al usuario
- En el modelo de datos normalizado, el documento de dirección contiene una referencia al documento del usuario

Ejemplo

```
{  
  _id: "joe",  
  name: "Joe Bookreader"  
}  
  
{  
  patron_id: "joe",  
  street: "123 Fake Street",  
  city: "Faketon",  
  state: "MA",  
  zip: "12345"  
}
```

Ejemplo (cont)

- Si los datos de la dirección se recuperan con frecuencia con la información del nombre y luego con la referencia, su aplicación debe emitir varias consultas para resolver la referencia
- El mejor modelo de datos sería incrustar los datos de dirección en los datos del usuario, como en el siguiente documento:

Ejemplo (cont)

```
{  
  _id: "joe",  
  name: "Joe Bookreader",  
  address: {  
    street: "123 Fake Street",  
    city: "Faketon",  
    state: "MA",  
    zip: "12345"  
  }  
}
```

Con el modelo de datos embebidos, su aplicación puede recuperar la información completa del usuario con una consulta

Modelo de relaciones *one-to-many* con documentos embebidos

- En base al ejemplo anterior:

```
{
  _id: "joe",
  name: "Joe Bookreader"
}

{
  patron_id: "joe",
  street: "123 Fake Street",
  city: "Faketon",
  state: "MA",
  zip: "12345"
}

{
  patron_id: "joe",
  street: "1 Some Other Street",
  city: "Boston",
  state: "MA",
  zip: "12345"
}
```

Modelo de relaciones *one-to-many* con documentos embebidos

- Si su aplicación recupera con frecuencia los datos de la dirección con la información del nombre, entonces su aplicación debe emitir varias consultas para resolver las referencias
- Un esquema más óptimo sería incrustar las entidades de datos de dirección en los datos del usuario, como en el siguiente documento:

Modelo de relaciones *one-to-many* con documentos embebidos

```
{
  _id: "joe",
  name: "Joe Bookreader",
  addresses: [
    {
      street: "123 Fake Street",
      city: "Faketon",
      state: "MA",
      zip: "12345"
    },
    {
      street: "1 Some Other Street",
      city: "Boston",
      state: "MA",
      zip: "12345"
    }
  ]
}
```

Modelo de relaciones *one-to-many* con referencias de documentos

- Considere el siguiente ejemplo que mapea relaciones de editor y libro. El ejemplo ilustra la ventaja de hacer referencia sobre documentos embebidos para evitar la repetición de la información del editor
- Embeber el documento del editor dentro del documento del libro llevaría a la repetición de los datos del editor, como muestran los siguientes documentos:

```
{
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}

{
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English",
  publisher: {
    name: "O'Reilly Media",
    founded: 1980,
    location: "CA"
  }
}
```

Modelo de relaciones *one-to-many* con referencias de documentos

- Para evitar la repetición de los datos del editor, use referencias y mantenga la información del editor en una colección separada de la colección de libros
- Cuando se usan referencias, el crecimiento de las relaciones determina dónde almacenar la referencia. Si el número de libros por editor es pequeño con un crecimiento limitado, a veces puede ser útil almacenar la referencia del libro dentro del documento del editor. De lo contrario, si el número de libros por editor no tiene límites, este modelo de datos llevaría a arreglos mutables y en crecimiento, como en el siguiente ejemplo:

```
{
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA",
  books: [123456789, 234567890, ...]
}

{
  _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English"
}

{
  _id: 234567890,
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
  pages: 68,
  language: "English"
}
```

Modelo de relaciones *one-to-many* con referencias de documentos

- Para evitar arreglos crecientes y mutables, almacene la referencia del editor dentro del documento del libro:

```
{
  _id: "oreilly",
  name: "O'Reilly Media",
  founded: 1980,
  location: "CA"
}

{
  _id: 123456789,
  title: "MongoDB: The Definitive Guide",
  author: [ "Kristina Chodorow", "Mike Dirolf" ],
  published_date: ISODate("2010-09-24"),
  pages: 216,
  language: "English",
  publisher_id: "oreilly"
}

{
  _id: 234567890,
  title: "50 Tips and Tricks for MongoDB Developer",
  author: "Kristina Chodorow",
  published_date: ISODate("2011-05-06"),
```

Agregaciones

- Lookup
- Sort
- Match
- Unwind
- Project

Lookup

```
db.orders.insert([
  { "_id" : 1, "item" : "almonds", "price" : 12, "quantity" : 2 },
  { "_id" : 2, "item" : "pecans", "price" : 20, "quantity" : 1 },
  { "_id" : 3 }
])
```

```
db.inventory.insert([
  { "_id" : 1, "sku" : "almonds", description: "product 1", "instock" : 120 },
  { "_id" : 2, "sku" : "bread", description: "product 2", "instock" : 80 },
  { "_id" : 3, "sku" : "cashews", description: "product 3", "instock" : 60 },
  { "_id" : 4, "sku" : "pecans", description: "product 4", "instock" : 70 },
  { "_id" : 5, "sku": null, description: "Incomplete" },
  { "_id" : 6 }
])
```

Lookup: Left outer join

```
db.orders.aggregate([
  {
    $lookup:
      {
        from: "inventory",
        localField: "item",
        foreignField: "sku",
        as: "inventory_docs"
      }
  }
])
```

Resultado

```
{
  "_id" : 1,
  "item" : "almonds",
  "price" : 12,
  "quantity" : 2,
  "inventory_docs" : [
    { "_id" : 1, "sku" : "almonds", "description" : "product 1", "instock" : 120 }
  ]
}
{
  "_id" : 2,
  "item" : "pecans",
  "price" : 20,
  "quantity" : 1,
  "inventory_docs" : [
    { "_id" : 4, "sku" : "pecans", "description" : "product 4", "instock" : 70 }
  ]
}
{
  "id" : 3,
```

Link de Referencia

- <https://docs.mongodb.com/manual/reference/operator/aggregation-pipeline/>

Ejemplo

- Obtener el teléfono y el número de cliente del cliente con nombre “Wanda” y apellido “Baker”
- `db.cliente.find({nombre: "Wanda", apellido: "Baker"}, {codigo_area: 1, nro_telefono:1}).pretty()`

Lógica de control

```
db.ideal.find(  
  {  
    fullyManaged: true,  
    security: 'built-in',  
    cloud:  
    {  
      $in: ['AWS', 'Azure', 'GCP']  
    }  
  },  
  {  
    _id: 0,  
    try: 1  
  }).forEach((doc) =>  
  {  
    printjson(doc);  
  });  
  
  {  
    'try': 'MongoDB Atlas Today'  
  }
```

Explain

The following example runs `cursor.explain()` in "executionStats" verbosity mode to return the query planning and execution information for the specified `db.collection.find()` operation:

```
db.products.find(  
  { quantity: { $gt: 50 }, category: "apparel" }  
)<img alt="copy icon" data-bbox="938 598 960 625"/>.explain("executionStats")
```